

Dispensa · Classe 2 · Informatica

# GitHub e il versioning

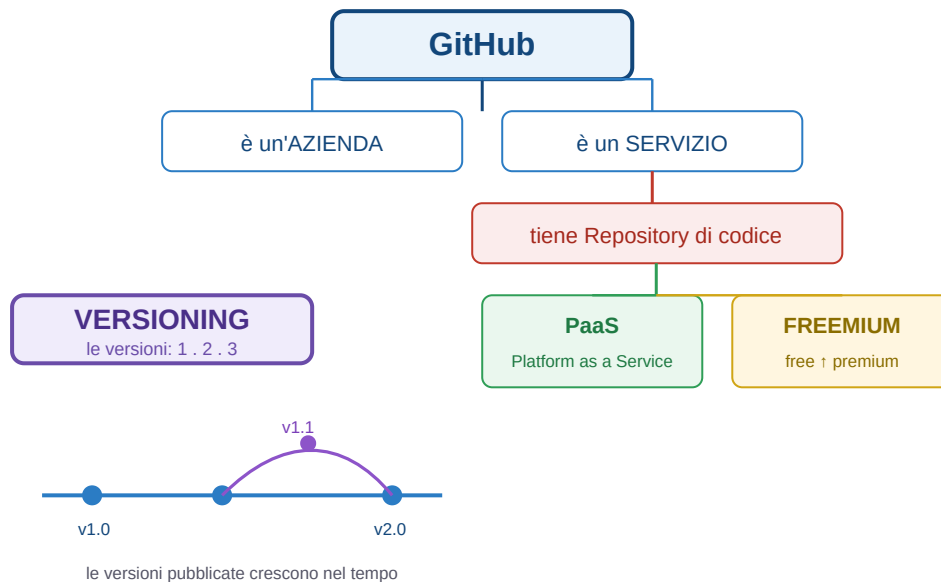
quello che abbiamo scritto insieme alla lavagna, spiegato per bene

Partiamo dallo schema della **nostra lavagna** e lo apriamo pezzo per pezzo: cos'è GitHub, che tipo di servizio è (PaaS, Freemium) e come funzionano le **versioni**. Con esempi e senza comandi da digitare.

Classe 2 · Informatica.

## 1. Partiamo dalla nostra lavagna

Ecco, ridisegnato in ordine, lo schema che abbiamo costruito insieme alla lavagna. Ogni riquadro è una tappa: nelle prossime pagine le apriamo una per una.



Lo schema della lezione: GitHub è azienda e servizio; tiene repository di codice; come servizio è una piattaforma (PaaS) col modello Freemium; e il progetto cresce per versioni (versioning).

## 2. GitHub: un'azienda e un servizio

Alla lavagna abbiamo scritto due parole sopra "GitHub": **azienda** e **servizio**. Vediamo perché tutte e due sono vere.

### È un'azienda

Dietro GitHub c'è una **società vera** — **GitHub, Inc.**, nata nel 2008 e dal 2018 di proprietà di **Microsoft**. Come ogni azienda deve mantenersi: ci arriviamo al punto "Freemium".

### È un servizio (online)

GitHub è un **sito** dove metti i tuoi progetti **online**: per tenerli al sicuro, mostrarli e lavorarci in gruppo. Non è un programma che installi col CD: lo usi dal **browser**.

**Attenzione a non confondere:** **Git** è lo **strumento** (la tecnologia che tiene la storia di un progetto, gratis e di nessuno); **GitHub** è il **servizio online** (l'azienda) dove quei progetti vivono in rete. Git è il **motore**, GitHub è il **garage online**.

## 3. Che cosa tiene: i repository di codice

Sulla lavagna, sotto "servizio", abbiamo scritto "**repository di codice**": è ciò che GitHub custodisce.

Un **repository** (in breve "repo") è la **cartella-progetto** con dentro i file **e tutta la loro storia**. Ogni salvataggio della storia si chiama **commit** (una "fotografia" del progetto con un messaggio). Se lavori in gruppo, puoi aprire un **branch** (una strada di lavoro parallela) e poi riunirlo con un **merge**.

**Esempio.** Il tuo gioco "SpaceRun" è un repository su GitHub: dentro c'è il gioco **e** il diario di tutte le modifiche. Puoi tornare a com'era la settimana scorsa in qualsiasi momento.

## 4. Che tipo di servizio è: PaaS

Alla lavagna: **PaaS = Platform as a Service**, cioè "piattaforma **come servizio**". Vuol dire che qualcuno ti mette a disposizione online una **piattaforma già pronta**, e tu la usi senza costruirti e mantenere i computer, i server e tutto il resto.

**Analogia della cucina.** Per aprire un ristorante puoi **costruirti la cucina da zero** (tanta fatica) oppure **affittare una cucina già attrezzata** e pensare solo a cucinare. GitHub è la cucina già pronta: tu porti il progetto, il resto lo mette lui.

### La famiglia "...aaS"

PaaS non è da solo: fa parte di una famiglia di servizi online, tutti con la sigla che finisce in **"aaS"** (as a Service). Cambia quanto lavoro fai tu:

| Sigla       | Nome                        | Cosa ti danno pronto                                    | Esempio                  |
|-------------|-----------------------------|---------------------------------------------------------|--------------------------|
| <b>IaaS</b> | Infrastructure as a Service | Solo i "muri e la corrente": server nudi.               | Amazon AWS, Google Cloud |
| <b>PaaS</b> | Platform as a Service       | La cucina già attrezzata: piattaforma pronta.           | GitHub, Netlify          |
| <b>SaaS</b> | Software as a Service       | Il piatto già servito: un programma pronto nel browser. | Gmail, Google Documenti  |

**Come ricordarli — la pizzeria.** **IaaS** = affitti il locale vuoto. **PaaS** = affitti la cucina attrezzata. **SaaS** = ti fai consegnare la pizza pronta. Più scendi, meno fatica fai.

## 5. Come si paga: il modello Freemium

Sulla lavagna: **Freemium = FREE ↑ PREMIUM**. È una parola nata unendo **free** (gratis) e **premium** (a pagamento). Significa: le funzioni **di base sono gratis** per tutti; se vuoi funzioni **in più**, paghi un abbonamento.

**Esempio che conosci.** Tanti giochi per telefono e app come Spotify funzionano così: gratis con qualche limite, a pagamento per togliere pubblicità o avere funzioni extra. **GitHub è uguale:** a te

studente basta e avanza la parte gratis.

**Ecco perché "azienda" e "Freemium" stanno insieme:** GitHub è un'azienda, quindi deve guadagnare; lo fa lasciando gratis l'uso normale e facendo pagare le funzioni avanzate a chi ne ha bisogno (aziende, team grandi).

## 6. Il versioning: le versioni 1.2.3

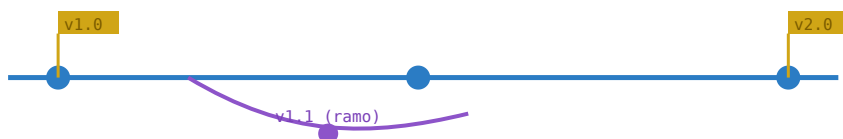
L'ultima parte della lavagna: **VERSIONING**, con il numero 1.2.3. Quando un progetto raggiunge un punto "buono", gli si dà un **numero di versione** fatto di tre cifre:

| Cifra | Nome  | Aumenta quando...                                                      |
|-------|-------|------------------------------------------------------------------------|
| 1     | major | cambi <b>tutto</b> , in modo grosso (magari incompatibile col vecchio) |
| 2     | minor | aggiungi una <b>funzione nuova</b> , ma il resto funziona ancora       |
| 3     | patch | correggi solo un <b>bug</b> , niente di nuovo                          |

**Esempio.** v1.0.0 prima versione giocabile → aggiungi i suoni: v1.1.0 → correggi un bug: v1.1.1 → rifai tutto da capo: v2.0.0.

### Il grafo che abbiamo disegnato

Alla lavagna avevamo disegnato la linea delle versioni con un ramo che si stacca e rientra. Ecco cosa racconta: il progetto va avanti su `main`, ogni tanto si pubblica una versione (le bandierine), e un ramo di lavoro si può staccare e poi riunire.



Da `v1.0` il progetto cresce; un ramo porta la `v1.1` e si riunisce; poi un grande cambiamento porta alla `v2.0`.

**Regola d'oro:** una versione già pubblicata **non si riusa mai**. Se cambi anche una sola cosa, prima **aumenti** il numero e poi pubblichi.

## 7. Riepilogo (la lavagna in tre righe)

1. **GitHub** è un'azienda e un **servizio** online dove tieni i tuoi **repository di codice** (con Git sotto, come motore).
2. Come servizio è una **piattaforma pronta (PaaS)**, della famiglia **...aaS**, e si paga in **Freemium** (base gratis, extra a pagamento).

3. Il progetto cresce per **versioni** ( 1.2.3 ): major, minor, patch — e una versione pubblicata non si tocca più.

Per vedere tante situazioni diverse con i grafi, guarda la scheda "Versioning — 11 casi d'uso".

# Compito: Git e Versioning

Classe 2 · Informatica · v1.2 · si scrive in Google Documenti e si consegna su Google Classroom · italiano

**Obiettivo:** dimostrare che hai capito **cos'è Git** e **come funzionano le versioni**, con parole tue. Conta la comprensione, non le frasi perfette.

## Ripasso lampo (prima di rispondere)

1. **Git** = la "macchina del tempo" del progetto: salva le modifiche (i **commit**) e ti fa tornare indietro.
2. **Git** è lo strumento; **GitHub** è il sito-servizio dove tieni il progetto online. GitHub è una **piattaforma (PaaS)**, della famiglia dei servizi **...aaS** (IaaS, PaaS, SaaS), e si paga in **Freemium** (base gratis, extra a pagamento).
3. **repository** = la cartella-progetto con la storia · **branch** = una strada parallela per provare · **merge** = unirla di nuovo al ramo principale.
4. **Versioning**: le versioni si scrivono **v1.2.3** (1 = cambiamento grosso, 2 = funzione nuova, 3 = correzione bug). Una versione pubblicata non si riusa più.

## Italiano

Apri un documento nuovo: nel browser vai su `docs.new` e dai un **titolo** (es. "Git e Versioning — Nome Cognome"). Rispondi alle tre parti.

### Parte 1 — Git (con parole tue)

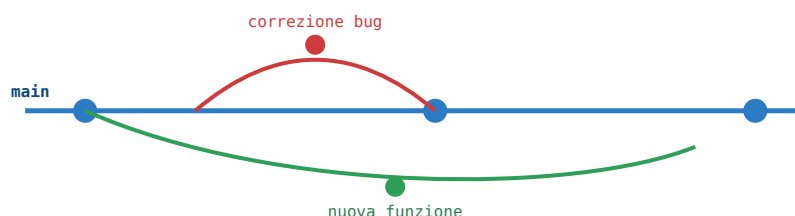
1. Cos'è **Git** e a che **problema** serve? (2-3 frasi)
2. Spiega questi termini con una frase ciascuno: **repository**, **commit**, **branch**, **merge**.
3. Qual è la **differenza tra Git e GitHub**?

### Parte 2 — Versioning (ragiona)

Le versioni si scrivono come **vX.Y.Z** (X = cambiamento grosso, Y = funzione nuova, Z = correzione bug). Un gioco è alla versione **v1.2.0**. Scrivi **quale sarà la nuova versione** in ciascun caso e **perché**:

1. Correggi un piccolo **bug**. → nuova versione: \_\_\_\_\_
2. Aggiungi un **nuovo livello** al gioco (le cose vecchie funzionano ancora). → \_\_\_\_\_
3. Riscrivi **tutto da capo** con un motore nuovo, incompatibile col vecchio. → \_\_\_\_\_

Poi guarda questo grafo e rispondi: **quanti branch** vedi oltre al main? **Quanti merge**? A cosa serviva il ramo rosso?



### Parte 3 — La tua esperienza

1. Racconta **cosa hai fatto oggi** con GitHub (es. creare l'account, entrare). Cosa è stato **facile**, cosa **difficile**?
2. Fai un **esempio tuo**, non del gioco: in quale altra situazione (una tesina, delle foto, un progetto) sarebbe utile avere la "storia delle versioni"?

Quando hai finito, **consegna** il documento su **Classroom**.

**Nota:** puoi usare la dispensa per ripassare, ma le risposte devono essere **con parole tue**. Conta soprattutto **saperlo spiegare a voce**.